



## **Guia #02: Comunicación en Sistemas Distribuidos (FASE 1)**

Jose Camilo Pérez Daza

Sebastián Fernando Revelo Meneses

Tomas Alejandro Santiago Reyes

Ingeniería de software, Universidad Manuela Beltrán

Sistemas distribuidos

Prof. Juan José Osorio Tabares

Bogotá D.C.

13 de agosto del 2026

## Tabla de Contenidos

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Descripción de la propuesta.....                                            | 3  |
| ¿Qué permite la aplicación? .....                                           | 4  |
| Justificación técnica de la propuesta .....                                 | 5  |
| Herramientas y tecnologías para utilizar .....                              | 6  |
| Comparación de características entre Cliente, Servidor 1 y Servidor 2 ..... | 7  |
| Diseño de la arquitectura .....                                             | 8  |
| Arquitectura de despliegue del sistema distribuido.....                     | 8  |
| Flujo de registro de información y de consulta entre servidores.....        | 9  |
| Recuperación y resincronización tras la caída de un servidor.....           | 10 |
| Perfil y roles de los integrantes .....                                     | 11 |
| Roles principales por integrante .....                                      | 11 |
| Próximos pasos (Fase 2).....                                                | 12 |

## Descripción de la propuesta

Como decía la guía, se tiene que hacer un servicio distribuido de registro y consulta de información, lo que nos pide es que sobre dos servidores y que tengan 2 servidores operativos, también contenido en contenedores Docker orquestados con docker-compose sobre una misma red interna.

- **Contenedor cliente:** Ubuntu seria como el consumidor de las dos API REST y pues el es el cliente y para acceder a la información, necesitaría de la consulta de alguno de los 2 servidores.
- **Contenedor servidor 1:** Debian (GNU/Linux), expone su API REST y administra su propia instancia de PostgreSQL (db1).
- **Contenedor servidor 2:** Fedora Linux, expone la misma API y administra su propia instancia de PostgreSQL (db2).

Es importante decir que cada servidor conserva su propio motor de base de datos en lugar de depender de una instancia externa compartida, pero los 2 servidores se comportan como réplicas de un mismo Deployment de Kubernetes:

- Deben mantener sincronía y ofrecer exactamente las mismas funcionalidades, para que cuando un servidor recibe un registro, lo guarda primero en su base local y luego lo réplica al otro.

Si uno de los dos se cae, el otro servidor funcionaria normal y seguiría dando los mismos datos con los que contaba y si se reactiva el nodo caído, este ejecuta un proceso de resincronización que con al nodo activo le pediría los registros que le falten hasta quedar consistente de nuevo.

El cliente, en sí, solo tendría un pequeño almacenamiento local en SQLite con un historial de sus propias comunicaciones (a qué servidor se conectó, cuándo y que paso), sin duplicar la información sustantiva que está en los servidores.

### **¿Qué permite la aplicación?**

- Registrar la información desde un cliente.
- Enviar la información al servidor que le corresponde.
- Consulta de la información almacenada.
- Mantener comunicación y sincronía constante entre los dos servidores (super importante).
- Identificar cuál de los dos servidores está atendiendo cada solicitud.
- Mostrar el estado de conexión de cada servidor.
- Seguir funcionando, aunque se conecte desde un servidor distinto, y aunque uno de los dos servidores esté caído temporalmente o fuera de servicio.
- Registrar en el cliente un historial local en SQLite de las solicitudes realizadas, sin duplicar los datos completos de los servidores.

*Al ejecutarse como contenedores Docker sobre el mismo host, Servidor 1 y Servidor 2 comparten en la práctica el kernel del sistema anfitrión:*

*La diferencia real entre ambos está en la distribución base de cada imagen, sus gestores de paquetes y sus bibliotecas de sistema y si se requieren kernels física y completamente distintos, cada servidor debería desplegarse en una máquina virtual independiente en lugar de un contenedor.*

### Justificación técnica de la propuesta

Para el diseño de la comunicación entre los servidores tenemos en cuenta cuatro criterios principales: Latencia, fiabilidad, desacoplamiento y tolerancia a fallos.

- **Latencia:** Cada servidor lee y escribe en su propia base de datos localmente, sin depender de una instancia externa compartida, el cliente a su vez se conecta directamente al servidor que esté disponible mediante HTTP/REST, lo que evita usar intermediarios y mantiene bajo el tiempo de respuesta.
- **Fiabilidad de la entrega:** Cada registro se guarda primero en la base de datos local del servidor que lo recibe, si el proceso de replicación hacia el otro nodo falla o no se encuentra disponible, el cambio ya se guardó y se marca como pendiente, lo que permite que ninguna escritura se pierda por una falla de red.
- **Desacoplamiento:** cada servidor es autónomo, con su propio motor de base de datos, su propia lógica de negocio y su propia API. Ósea que ninguno depende de que el otro esté disponible ni de un componente externo compartido a la hora de atender solicitudes.
- **Tolerancia a fallos de red:** Si la comunicación entre los servidores se corta, cada uno puede continuar funcionando, escribiendo y leyendo datos en su propia BD local sin afectar la experiencia al usuario, de forma que cuando se reestablezca la comunicación se resincronizan los nodos con los cambios realizados mientras se presentó la falla.

A todo esto, se le suma la sincronía de tipo Kubernetes que se plantea, cada servidor mantiene su propio estado local, pero lo replica constantemente con el otro nodo, de forma que ambos mantienen los mismos datos mientras estén sincronizados, como nota adicional, a diferencia

de un clúster de Kubernetes real, aquí la convergencia no depende de un almacenamiento externo compartido sino de la replicación activa-activa entre los dos nodos.

### **Herramientas y tecnologías para utilizar**

- **Lenguaje / framework:** Python, con Flask y/o FastAPI para exponer los servicios como API REST en ambos servidores.
- **Protocolo de comunicación:** REST sobre HTTP para cliente-servidor, y TCP/IP para heartbeat y replicación de estado entre Servidor 1 y Servidor 2.
- **Base de datos:** PostgreSQL, con una instancia local independiente por servidor (db1 y db2), sincronizadas entre sí mediante replicación aplicativa mediante el canal TCP/IP.
- **Almacenamiento local del cliente:** SQLite, para conservar un historial mínimo de las solicitudes del cliente (servidor que respondió, marca de tiempo, resultado), sin replicar los datos completos de los servidores.
- **Diagramación:** PlantUML, para construir los diagramas de arquitectura y de secuencia como código versionable junto con el resto del proyecto en el repositorio.
- **Contenedores:** Docker y docker-compose, para levantar el cliente, los dos servidores y sus respectivas bases de datos sobre una misma red interna.
- **Control de versiones:** Git / GitHub para el código fuente compartido.
- **Entorno de desarrollo:** VS Code como editor principal.

Se eligió Flask/FastAPI porque permite construir rápidamente servicios REST livianos, es compatible con Debian y Fedora sin cambios en el código, y facilita implementar tanto el registro y consulta de datos como la replicación entre servidores. PostgreSQL se mantiene como motor único de cada servidor por su soporte transaccional ACID; Docker Compose permite levantar todos los servicios con un único comando sobre la misma red interna.

### Comparación de características entre Cliente, Servidor 1 y Servidor 2

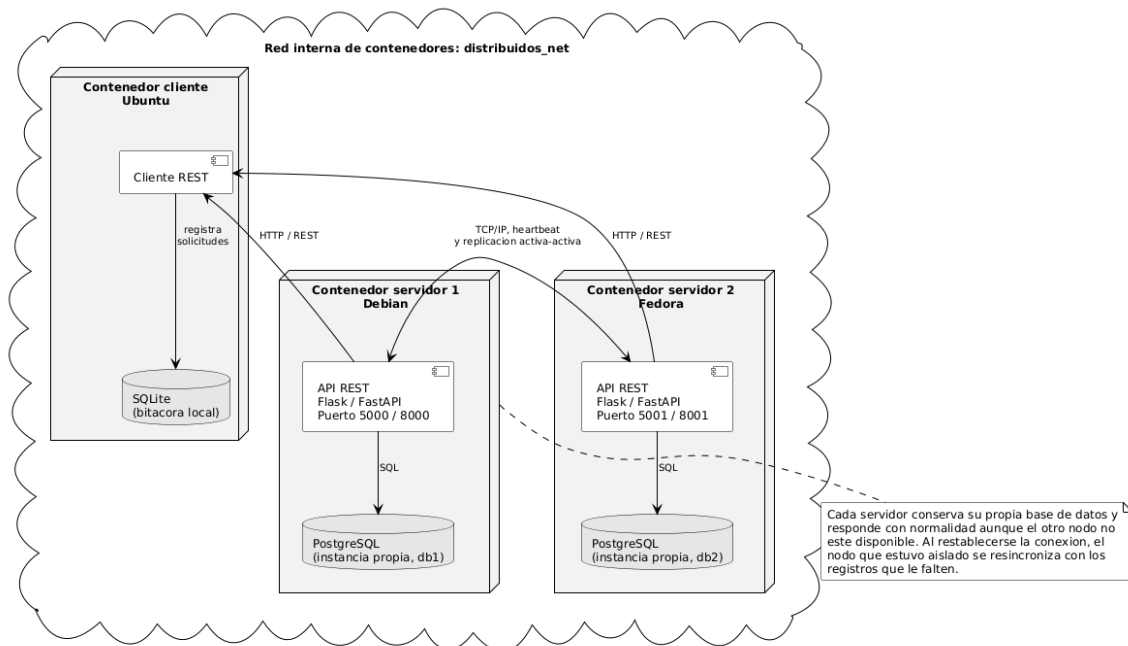
| Característica                         | Cliente                                     | Servidor 1                               | Servidor 2                               |
|----------------------------------------|---------------------------------------------|------------------------------------------|------------------------------------------|
| <b>Sistema operativo (imagen base)</b> | Ubuntu                                      | Debian (GNU/Linux)                       | Fedora Linux                             |
| <b>Arquitectura</b>                    | x86_64                                      | x86_64                                   | x86_64                                   |
| <b>Dirección IP</b>                    | Por definir en red interna                  | Por definir en red interna               | Por definir en red interna               |
| <b>Puerto / rol de red</b>             | No expone puerto, sería nada más consumidor | 5000 (Flask) / 8000 (FastAPI)            | 5001 (Flask) / 8001 (FastAPI)            |
| <b>Lenguaje / runtime</b>              | Python 3 (cliente de pruebas)               | Python 3 + Flask o FastAPI               | Python 3 + Flask o FastAPI               |
| <b>Base de datos local</b>             | SQLite (Historial de comunicaciones)        | PostgreSQL (instancia propia, db1)       | PostgreSQL (instancia propia, db2)       |
| <b>Imagen base Docker</b>              | ubuntu:22.04                                | debian:stable                            | fedora:latest                            |
| <b>Rol en el sistema</b>               | Consume las API REST de ambos servidores    | Nodo activo, atiende registro y consulta | Nodo activo, atiende registro y consulta |

*Nota. Las direcciones IP definitivas se confirmarán al configurar la red interna que se tendrá cuando estemos ya en la etapa de implementación.*

## Diseño de la arquitectura

El siguiente diagrama, muestra el cliente y los dos servidores, cada uno con su propia base de datos, la red interna de contenedores y los protocolos de comunicación entre cada componente.

### Arquitectura de despliegue del sistema distribuido

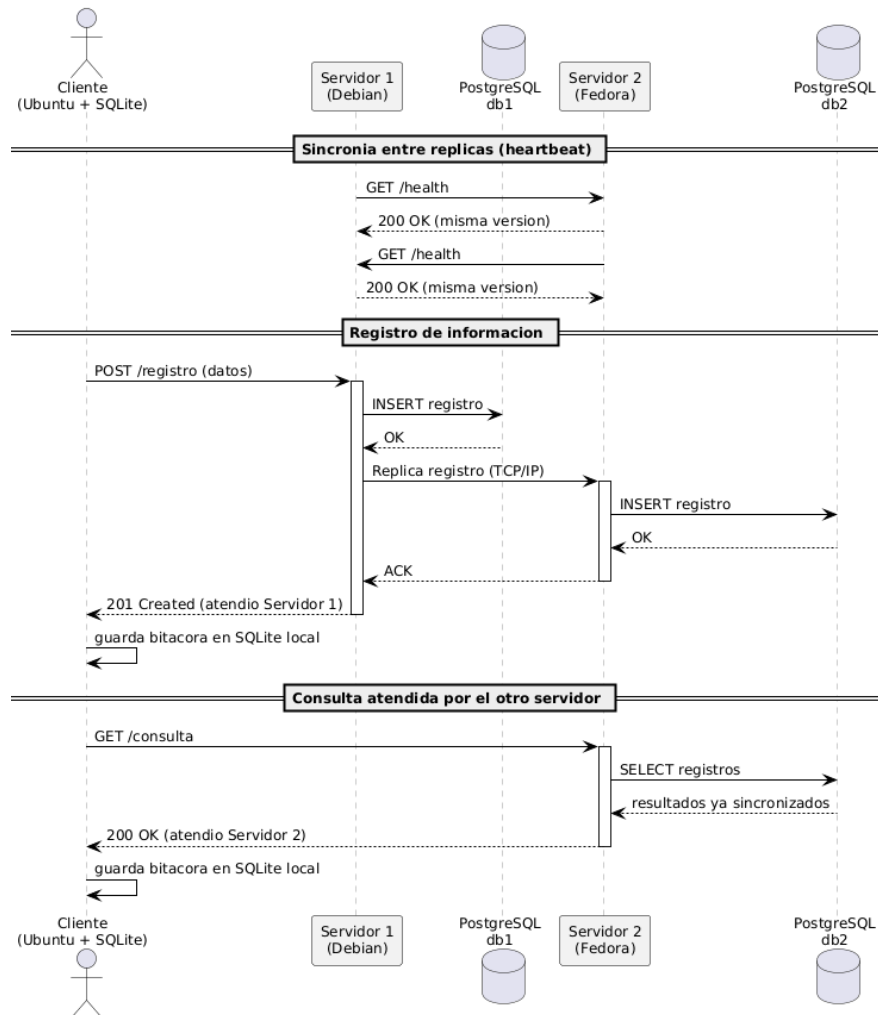


*Nota. Realizado por Claude según las peticiones que le dimos*

El siguiente diagrama de secuencia ilustra el heartbeat de sincronía entre los dos servidores y el flujo de información para un registro y una consulta atendida por el servidor par, evidenciando cómo se identifica el servidor que atiende cada solicitud.



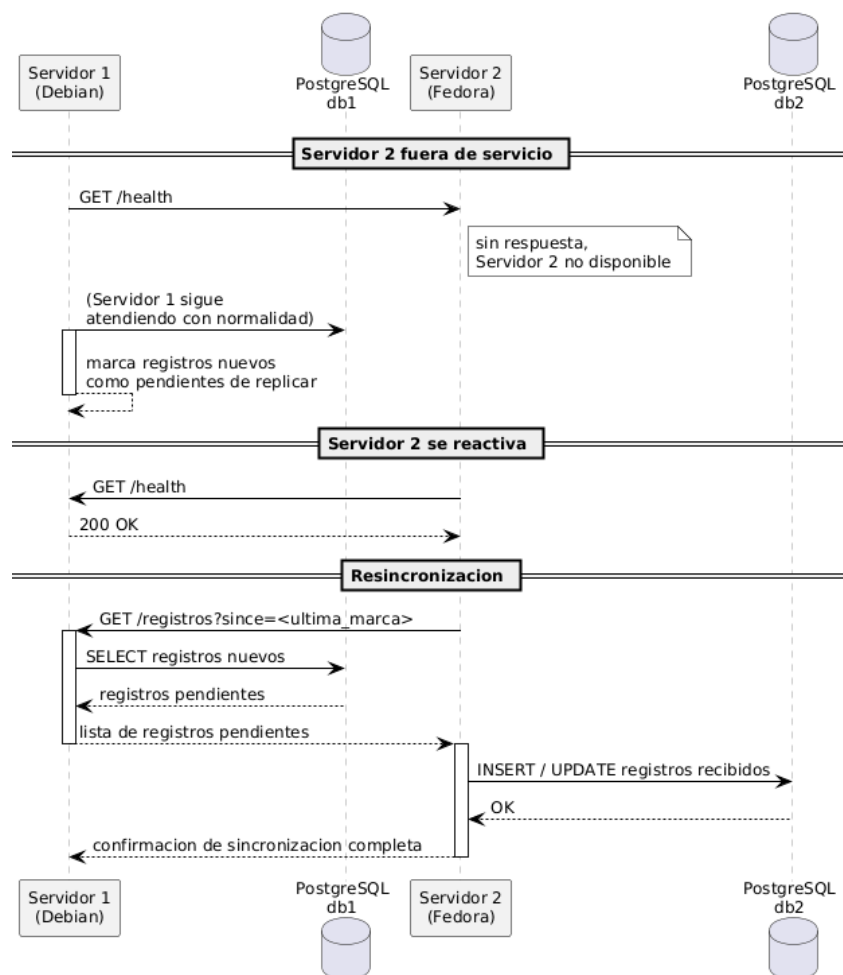
## Flujo de registro de información y de consulta entre servidores



*Nota. Realizado por Claude según las peticiones que le dimos*

El siguiente diagrama de secuencia muestra que pasaría en un escenario de falla y recuperación, donde el Servidor 2 queda fuera de servicio, el Servidor 1 continúa atendiendo con normalidad y al reactivarse el Servidor 2 se resincroniza con los registros que le faltan.

## Recuperación y resincronización tras la caída de un servidor



*Nota. Realizado por Claude según las peticiones que le dimos*

## Perfil y roles de los integrantes

### Roles principales por integrante

| Integrante       | Rol principal                                                                 | Aporte y herramientas                                                                                                                                  |
|------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tomás</b>     | Desarrollo del Servidor 1<br><br>(contenedor Debian) con<br><br>Flask/FastAPI | Configuración de Debian,<br><br>implementación de la API REST y de su<br><br>instancia local de PostgreSQL (db1).                                      |
| <b>Sebastián</b> | Desarrollo del Servidor 2<br><br>(contenedor Fedora) con<br><br>Flask/FastAPI | Configuración de Fedora,<br><br>implementación de la API REST y de su<br><br>instancia local de PostgreSQL (db2).                                      |
| <b>José</b>      | Replicación, orquestación e<br><br>integración                                | Mecanismo de replicación y<br><br>resincronización entre db1 y db2, docker-<br><br>compose de los servicios, diagramas y<br><br>manual de instalación. |

*Nota. Los tres trabajaremos en conjunto en todas las etapas (análisis, diseño, implementación y pruebas), pero cada uno lidera la parte indicada avanzar a la vez; hecho por Claude*

### **Próximos pasos (Fase 2)**

- Escribir el docker-compose.yml con los servicios de cliente, los dos servidores y sus respectivas bases de datos.
- Implementar la lógica de replicación entre db1 y db2, y el proceso de resincronización tras una caída.
- Configurar la red interna de docker-compose, direcciones IP y puertos definitivos.
- Documentar instalación, configuración y pruebas de comunicación y de recuperación ante fallos.
- Grabar el video explicativo mostrando el funcionamiento normal y el escenario de falla y recuperación.